Randy Chan Yi Zhong

AI Software Engineer · AI-Native Development, AI-Driven Workflows and Multi-Agent Systems · End to End, Production First

Singapore · randychan_92@outlook.com · linkedin.com/in/randychan112 · github.com/randyharrogates

Summary

AI software engineer, 8+ years in software, building AI-native products, AI-driven workflows and multi-agent systems end to end, from requirements and prototypes to production and on-call. Multi-agent systems: specialist agents orchestrated in LangGraph with tool calling, retrieval with hybrid search and reranking, a judge model checking every section against its sources, and human approval before high-risk decisions or writes. AI-driven workflows: event-driven pipelines, MCP servers that let agents act as the signed-in user, and long-running agents with approval gates and feedback loops. AI-native development: Claude Code and Codex run in parallel every day, in separate git worktrees, with loops that run until the tests pass; I review and stand behind every merge, and moving a team onto this way of working took releases from about 4 a month to about 20. Full stack in Python (FastAPI), TypeScript (Node.js, React, Next.js) and Java, on AWS, Azure and Cloudflare. Now Head of Technology and AI at The Assembly Place, leading a small team.

Experience

The Assembly Place Aug 2026 – Present

Head of Technology and AI *Singapore*

- Lead a small technology team that builds and runs the company's software products: an in-house CRM used daily by operations staff, its admin app and APIs, and an internal AI-agent console. Also run company IT, including security, AWS IAM and audit.
- Run an AI-first development workflow: Claude Code and Codex work in parallel from shared project standards and reusable skills, one planning and reviewing while the other implements and tests; most new code is written this way, and people stay accountable for every merge.
- Agent access without exposed credentials: agents reach company data through an MCP server over the CRM where every request runs as the signed-in user (OAuth, per-user tokens) with a confirmation step before any write; the agent console connects to company systems through MCP connectors and delegates only to approved sub-agents.
- Loop engineering: long-running agent loops work through Jira tickets and babysit pull requests; reviewer agents send work back until tests and checks pass; parallel agents run in separate git worktrees; team skills, hooks and CLAUDE.md standards steer every coding agent.
- Built the internal AI-agent console end to end in TypeScript: a Node.js (Fastify) API on PostgreSQL and a React web app with live updates over WebSockets.
- Run the AWS infrastructure as code (Terraform) with deployment safeguards, scheduled cost controls and monitoring; releases are tested in staging against a written checklist, and production data changes run from rehearsed runbooks.

TrustPlus AI (Fintech SaaS) Mar 2025 – Aug 2026

Senior AI Software Engineer *Singapore*

- Designed and built most of the credit-risk and KYB SaaS product, from requirements workshops and prototypes with business users to production; worked across the entire stack (React frontend, Python FastAPI APIs, MongoDB, AI pipelines, Docker, GitHub Actions CI/CD). Analyst turnaround fell from 6–24 hours to 5–60 minutes.
- Scaling: MongoDB vector search hit its I/O limits under parallel load and upgrading the tier cost too much, so I moved search to in-memory FAISS and parallelised it; measured on LangFuse retrieval spans over the same test cases, latency fell about 85% (700–1,400 ms to 100–250 ms).
- Production incident: traced and resolved a memory failure in the event-driven pipeline, where concurrent heavy jobs exhausted container memory and redelivered messages restarted the same work; per-step timing and error fields in the run state located the fault and now drive clear user-facing error messages.
- Event-driven backend on Azure Service Bus (14 consumers) and Container Apps, infrastructure in Terraform; idempotency-aware retries and atomic upserts against concurrent callers; a LangChain multi-provider LLM layer (OpenAI, Anthropic, Google, Amazon Bedrock fallback) with rate-limit handling and per-run token and cost tracking.
- AI features in the product (LangGraph, LangChain): 8 specialist LangGraph research agents that read financial statements, SEC EDGAR filings and web results and write a cited credit memo; RAG with hybrid retrieval and reranking; an LLM-as-a-judge check on every section; human approval on high-risk decisions; a chatbot that routes each request to a fast edit path or a retrieval path so users can question and edit a finished memo.
- Built an MCP server (8 tools, OAuth and JWT sign-in as the acting user) so AI assistants act only with that user's permissions, and generated a 14-skill Claude Code pack from the OpenAPI spec so new endpoints become agent tools automatically.
- Improved how the team ships without being asked: led the move from Cursor to Claude Code across development, design and product work, with shared CLAUDE.md standards and skills I wrote for code review, change planning, test review and changelogs; releases rose from about 4 a month to about 20 while review standards held.
- On call for production and the escalation point for user-reported issues: triaged from LangFuse traces and CloudWatch logs, reproduced and fixed with the engineers involved, and fed the lessons back into tests and team skills.

BGC Group (Defence-sector client), *Senior Software Engineer (AI Application Specialist), Contract* Nov 2024 – Mar 2025

- Delivered an enterprise RAG knowledge assistant for a Singapore defence-sector client on a fully on-premise stack, because the data could not leave the environment: built the React frontend and the full-stack deployment, with Docker-based CI/CD across development, staging and production; worked directly with the client's business users on discovery and fit.
- Designed the hand-off so complex requests move from the assistant to a human agent.

AI Singapore, *AI Engineer* Jan 2024 – Oct 2024

- Delivered a production NLP pipeline for a healthcare environment: five models for named-entity recognition and event prediction (including a fine-tuned BioClinicalBERT), served with FastAPI and Docker, with Kedro pipelines for reproducible training and evaluation; made predictions explainable to clinicians with Explainable Boosting Machines.
- Mentored 12 apprentices across two cohorts in NLP and LLM techniques.

NCS Pte. Ltd., *Business Systems Analyst (Backend Automation & Integration)* Oct 2022 – Dec 2023

- Improved operational productivity by 50% for a nation-wide public healthcare web application with a Python automation suite that validates configuration changes across environments, replacing a manual process.
- Ran SQL data migrations between systems with 99.9% data integrity, documented system architecture and BPMN workflows for cross-functional teams, and advised on PDPA compliance.

Bitopia Technology, *Full-Stack Software Engineer* Jan 2022 – Oct 2022

- Ran client requirements workshops and pre-sales scoping, then built REST microservices and React dashboards for logistics and real-estate platforms, integrating mapping, carrier and payment APIs behind real-time tracking with PostgreSQL.

Napshot Pte. Ltd., *Solutions & Software Engineer* Jan 2019 – Dec 2021

- Worked across 3+ concurrent SME client engagements (F&B, hospitality, HR): requirements workshops, solution design and custom platforms (React, Vue, Flask) that replaced paper and spreadsheet processes, taken through to go-live; ran the cloud infrastructure, monitoring and incident response for client deployments.

Singapore Army (SAF), *Combat Engineer, Regular Force* 2012 – 2018

- Led and trained personnel up to company level (100 to 200), and ran user-acceptance testing and feedback cycles on equipment and systems with operational end users.

Projects

Jarvis, my own long-running agent harness on Claude Code, running 24/7

- Explored OpenClaw and ClaudeClaw early in 2026, then built my own: one long-lived Claude Code session woken by launchd jobs and local Gmail, calendar and chat watchers through an event queue with cursors and idempotent handling, so every pass is safe to repeat.
- Guardrails: a pre-tool-use hook gates outbound and destructive actions by where the turn came from, relays approvals to my phone, and treats all email and web content as untrusted data; feedback loops turn my corrections into standing rules that later runs apply.
- 27 custom skills, a Python CLI with 46 test files and 9 architecture decision records; built by planning with Claude and implementing with Codex.

TWC, browser-only agent with its own agent loop, live at randyharrogates.github.io/twc

- Wrote the agent loop from scratch: up to 32 iterations with streamed phase labels, permission-gated tool dispatch, conversation pruning at 60% of the context budget, and a plan mode that removes mutating tools from the schema rather than relying on the prompt.
- One client interface over Anthropic, OpenAI and local OpenAI-compatible servers, each a hand-written fetch call with no vendor SDK; API keys stay in a passphrase-encrypted vault in the browser (PBKDF2 and AES-GCM), with rate limits and spend caps. React, TypeScript, Zod.

Leeseidon, agentic investment tracker, live at leeseidon.com

- Multi-turn agents extract trades from CSV, PDF and images through a model-agnostic tool executor with streaming and retries; an encryption-blind sync server stores only ciphertext; one TypeScript core powers the web app and a CLI that ships Claude Code skills.

Kestrel, multi-agent review of insurance policy documents

- Four specialist extraction agents on the open-source claw-code agent runtime: per-field confidence and citations, deterministic compliance gates, required human confirmation and an audit trail; SSE streaming to a React front end.

Skills

AI-native development: Claude Code and Codex in parallel, git worktrees, run-until-green and review loops, custom skills and hooks, CLAUDE.md standards, AI-assisted code review, change planning and test review, Cursor
Agents: agent harnesses and loops, tool calling, MCP servers and clients, OAuth-scoped agent access, human-in-the-loop approval, LangGraph, LangChain, Claude and OpenAI SDKs, RAG, LLM-as-a-judge, structured outputs
Full stack: TypeScript, React, Next.js, Node.js (Fastify, Express), Python (FastAPI, Pydantic), Java (Spring Boot), REST and OpenAPI, WebSockets, Server-Sent Events, PWA
Production fundamentals: pytest, Playwright, LangFuse, CloudWatch, Sentry, on-call; Docker, Terraform, GitHub Actions CI/CD; AWS, Azure, Cloudflare Workers; PostgreSQL, MongoDB, Redis

Education & Certifications

Singapore University of Social Sciences, *B.Sc. Information & Communication Technology, Minor in Data Science (GPA 4.2/5)*

- Certified Associate AI Engineer (CAIE), AI Professionals Singapore · Entry Certificate in Business Analysis (ECBA), IIBA.